UNIVERSITY OF TEHRAN
Electrical and Computer Engineering Department
Object-Oriented Modeling of Electronic Circuits, Spring 1405
Final Take Home Exam

Name **Date**

Description:

Signal processing plays a significant role in embedded system design. In this assignment, you will enhance the SAYAC embedded system by adding an FIR filter accelerator to speed up signal processing.

Part 1: FIR Filter: Implement a FIR filter with 31 taps. Note that the filter operates in two phases:

1. **Initialization phase:** The filter loads the provided coefficients for FIR processing.
2. **Processing phase:** The filter performs FIR processing based on the input data and the loaded coefficients.

You should take the following requirements into account:

1. **Module hierarchy**: Your module must include the following structure:
   - A coefficients memory map register with an interface that matches the existing SAYAC embedded system bus. (Choose the memory-map range on your own.)
   - An input interface that reads data from a buffer using a SystemC predefined FIFO channel.
   - An output interface that writes data to a buffer using a SystemC predefined FIFO channel.
   - An interrupt port to interrupt the processor.
2. **Implementation approach**: Implement the filter as a **bus functional model**. From the bus perspective, everything should appear exactly like real hardware, but internally the behavior is implemented in C++ code.
3. **Processing phase behavior**: In the processing phase, the module should read a chunk of 512 data samples, perform the computation, interrupt the processor, and then allow the results to be read back.
4. **Data continuity**: Note that the data blocks are continuous and the filter only process a given chunk of it. Therefore, you must store any necessary overlapping data from each 512-sample chunk to be used with the next chunk.

Part 2: DMA: Add a DMA controller to the design to accelerate data transfers and exploit concurrency. Connect the DMA to the proposed FIR filter module.

You should take the following requirements into account:

1. Your module is a **memory-mapped accelerator** and includes the following memory-mapped registers:
   - **Base-Address**
   - **Config** (contains the enable bit, burst count, and burst size)
     - Do not forget the direction bit, which indicates whether the transfer is from accelerator to memory or from memory to accelerator.
   - **Status**

2. Your module should provide input and output interfaces for using the SystemC predefined FIFO channel to load and store data to external buffer.
3. Connect DMA to the FIR filter accelerator using the SystemC predefined FIFO.

Part 3: Bus structure: In this part, you will complete the system skeleton by connecting the DMA to the bus and to the accelerator.

You should take the following requirement into account:

1. Choose a memory-mapped range address for the DMA. Try not to modify the existing memory map, simply add a new address range for the DMA.

Part 4: Processor: In this part, you will use a **bracketing** approach to model your processor and simulate its functionality (the computation and the synchronization i.e. scheduler). Provide C code to manage the system and use your peripheral to perform the required task.

You should take the following requirements into account:

1. The processor should behave exactly like a real processor hardware from the bus perspective, performing all handshaking and memory operations as actual hardware would. However, internally it contains only C code (the bracket).
2. Note that the only difference between this simulated processor and real hardware (or even ISS) is that there is no instruction interface and no instruction fetching in this model.

Part 5: Channel: In this part, you will use channels to abstract the wired bus. All handshaking and communication signals must be abstracted within channels.

You should take the following requirements into account:

1. Use the channel discussed in class, instead of the wired bus.

To Do:

Pre work:

1. A sound file has been provided for you. It contains an annoying sharp noise.
2. Use the provided C++ code to convert the .wav file to a .txt file.
3. Place this .txt file in memory as initialized data. (In a real system, you would need to capture this data, but for this assignment, simply assume it already exists in memory.)
4. Take the coefficients from coefficients.txt and place them in memory as well.

Work: (Use the processor only for computation and scheduling.)

UNIVERSITY OF TEHRAN
Electrical and Computer Engineering Department
Object-Oriented Modeling of Electronic Circuits, Spring 1405
Final Take Home Exam

1. Use the coefficients to initialize the FIR filter.
2. Filter the stored data and store the result in memory.
3. After filtering, use the processor to create a 2x version and a 0.5x version of the sound:
    - **a. For 2x:** Read the data from memory, remove every other sample (one in the middle), and store the remaining data. (The resulting size should be half.)
    - **b. For 0.5x:** Read the data from memory, compute the mean of every two consecutive samples, and insert the result between them. Then store the data in memory. (The resulting size should be doubled.)

Post work:

1. Extract the three sets of sound data from memory into three separates .txt files.
2. Use the provided code to convert each .txt file to .wav format so you can listen to them. You should listen to the clean sound.

Available to You:

- Noisy sound (Noisy.wav)
- Sound-to-text converter (wav_to_txt.cpp)
- Text-to-sound converter (txt_to_wav.cpp)
- Coefficients (coefficients.txt)
- A C++ example of the filtering flow (fir_filter.cpp)
- SAYAC embedded system
- An example for MLP using bracketing
- Last year Home Work MLP
- A section of Embedded Core Design with FPGAs, 2007, McGraw-Hill, by Zain Navabi

Requested from You:

- Clean sound
- Clean 2× sound
- Clean 0.5× sound
- Proposed system code
- Illustration of the system (hand-drawn, draw.io, Visio, etc.)
- A report showing the flow of the work

Delivered phase:

1. A draft sketch of the system design, along with the DMA and Accelerator BFM code.
2. A system comprising the DMA, BFM, Bus structure and the software program.
3. A system comprising Channel bus structure.

UNIVERSITY OF TEHRAN
Electrical and Computer Engineering Department
Object-Oriented Modeling of Electronic Circuits, Spring 1405
Final Take Home Exam

Attention:

Make a *PDF* file of your report and submit it to the course site. Also, compress all files and documents mentioned in the *Deliverables* section into a *zip* file and upload the generated file. The name of the zip file must be in this format "***YourFirstName-YourLastName-FINAL***". In addition, use exactly this phrase "***Submitting FINAL***" as the subject of your email.